

Joins And Operations :

Program:

```
1  -- Step 1: Create Database
2  • CREATE DATABASE joins_operations_db;
3  • USE joins_operations_db;
4  -- Step 2: Create Tables
5  • CREATE TABLE class(
6      id INT,
7      name VARCHAR(30)
8  );
9
10 • CREATE TABLE class_info(
11     id INT,
12     address VARCHAR(30)
13 );
14 -- Step 3: Insert Data
15 • INSERT INTO class VALUES
16     (1,'Tejaswini'),
17     (2,'Praneeth'),
18     (3,'Sahasra'),
19     (4,'Pavani'),
20     (5,'Nikila');
```

Output:

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	14:29:55	CREATE DATABASE joins_operations_db	1 row(s) affected	0.016 sec
✓ 2	14:29:57	USE joins_operations_db	0 row(s) affected	0.000 sec
✓ 3	14:29:59	CREATE TABLE class(id INT, name VARCHAR(30))	0 row(s) affected	0.016 sec
✓ 4	14:30:02	CREATE TABLE class_info(id INT, address VARCHAR(30))	0 row(s) affected	0.016 sec
✓ 5	14:30:04	INSERT INTO class VALUES (1,'Tejaswini'), (2,'Praneeth'), (3,'Sahasr...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec

Program:

```
INSERT INTO class_info VALUES
(1,'DELHI'),
(2,'MUMBAI'),
(3,'CHENNAI'),
(7,'NOIDA'),
(8,'PANIPAT');
-- Step 4: CROSS JOIN
SELECT *
FROM class
CROSS JOIN class_info;
```

Output:

The screenshot displays a database IDE interface. The top section shows a 'Result Grid' with 25 rows of data resulting from a CROSS JOIN between 'class' and 'class_info'. The data is organized into four columns: 'id', 'name', 'id', and 'address'. The rows show combinations of student names (Nikila, Pavani, Sahasra, Praneeth, Tejaswini) and city names (DELHI, MUMBAI, CHENNAI). The bottom section shows an 'Action Output' log with 8 entries, detailing the execution of SQL commands and their results.

id	name	id	address
5	Nikila	1	DELHI
4	Pavani	1	DELHI
3	Sahasra	1	DELHI
2	Praneeth	1	DELHI
1	Tejaswini	1	DELHI
5	Nikila	2	MUMBAI
4	Pavani	2	MUMBAI
3	Sahasra	2	MUMBAI
2	Praneeth	2	MUMBAI
1	Tejaswini	2	MUMBAI
5	Nikila	3	CHENNAI
4	Pavani	3	CHENNAI
3	Sahasra	3	CHENNAI
2	Praneeth	3	CHENNAI

#	Time	Action	Message	Duration / Fetch
3	14:29:59	CREATE TABLE class(id INT, name VARCHAR(30))	0 row(s) affected	0.016 sec
4	14:30:02	CREATE TABLE class_info(id INT, address VARCHAR(30))	0 row(s) affected	0.016 sec
5	14:30:04	INSERT INTO class VALUES (1,'Tejaswini'), (2,'Praneeth'), (3,'Sah...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
6	14:31:14	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMBAI'), (3,'C...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
7	14:31:20	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
8	14:32:14	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	id	name	id	address
	4	Pavani	3	CHENNAI
	3	Sahasra	3	CHENNAI
	2	Praneeth	3	CHENNAI
	1	Tejaswini	3	CHENNAI
	5	Nikila	7	NOIDA
	4	Pavani	7	NOIDA
	3	Sahasra	7	NOIDA
	2	Praneeth	7	NOIDA
	1	Tejaswini	7	NOIDA
	5	Nikila	8	PANIPAT
	4	Pavani	8	PANIPAT
	3	Sahasra	8	PANIPAT
	2	Praneeth	8	PANIPAT
	1	Tejaswini	8	PANIPAT

Result 2 ×

Program:

```
32      -- Step 5: INNER JOIN
33      SELECT *
34      FROM class
35      INNER JOIN class_info
36      ON class.id = class_info.id;
```

Output:

The screenshot shows a database management interface. At the top, there's a 'Result Grid' with the following data:

	id	name	id	address
1	1	Tejaswini	1	DELHI
2	2	Praneeth	2	MUMBAI
3	3	Sahasra	3	CHENNAI

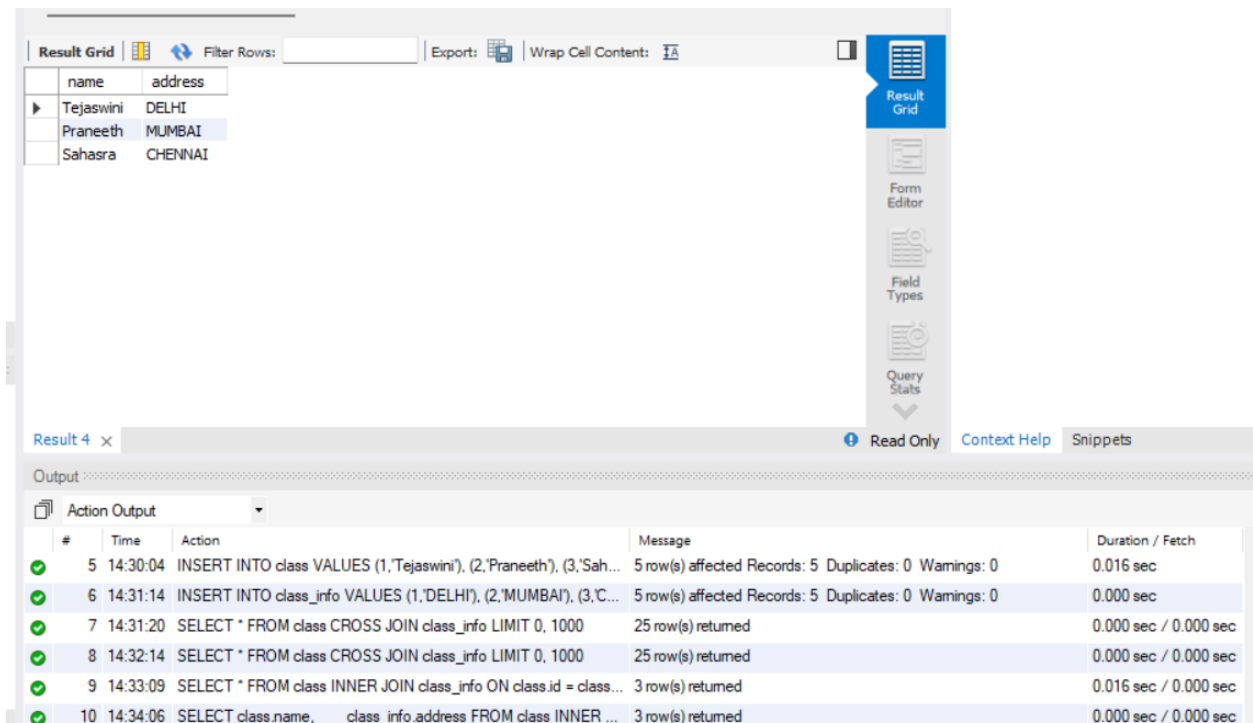
Below the result grid, there's a vertical toolbar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. At the bottom, there's an 'Output' section with a tab labeled 'Action Output'. It contains a log of database actions:

#	Time	Action	Message	Duration / Fetch
4	14:30:02	CREATE TABLE class_info(id INT, address VARCHAR(30))	0 row(s) affected	0.016 sec
5	14:30:04	INSERT INTO class VALUES (1,'Tejaswini'), (2,'Praneeth'), (3,'Sah...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
6	14:31:14	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMBAI'), (3,'C...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
7	14:31:20	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
8	14:32:14	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
9	14:33:09	SELECT * FROM class INNER JOIN class_info ON class.id = class...	3 row(s) returned	0.016 sec / 0.000 sec

Program:

```
37  -- Step 6: INNER JOIN Selected Columns
38  •  SELECT class.name,
39         class_info.address
40  FROM class
41  INNER JOIN class_info
42  ON class.id = class_info.id;
```

Output:



The screenshot displays a database management interface. At the top, a 'Result Grid' shows the output of the SQL query, listing three rows with columns 'name' and 'address':

name	address
Tejaswini	DELHI
Praneeth	MUMBAI
Sahasra	CHENNAI

Below the result grid, the 'Action Output' table provides a detailed log of the database actions:

#	Time	Action	Message	Duration / Fetch
5	14:30:04	INSERT INTO class VALUES (1,'Tejaswini'), (2,'Praneeth'), (3,'Sah...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
6	14:31:14	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMBAI'), (3,'C...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
7	14:31:20	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
8	14:32:14	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
9	14:33:09	SELECT * FROM class INNER JOIN class_info ON class.id = class...	3 row(s) returned	0.016 sec / 0.000 sec
10	14:34:06	SELECT class.name, class_info.address FROM class INNER ...	3 row(s) returned	0.000 sec / 0.000 sec

Program:

```
43      -- Step 7: NATURAL JOIN
44  •    SELECT *
45      FROM class
46      NATURAL JOIN class_info;
```

Output:

The screenshot displays a database IDE interface. At the top, a 'Result Grid' shows the output of a query with three columns: 'id', 'name', and 'address'. The data rows are:

	id	name	address
▶	1	Tejaswini	DELHI
	2	Praneeth	MUMBAI
	3	Sahasra	CHENNAI

Below the result grid, the 'Output' pane shows a log of database actions. The log includes a table with columns: '#', 'Time', 'Action', 'Message', and 'Duration / Fetch'.

#	Time	Action	Message	Duration / Fetch
✓ 6	14:31:14	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMBAI'), (3,'C...)	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
✓ 7	14:31:20	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
✓ 8	14:32:14	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
✓ 9	14:33:09	SELECT * FROM class INNER JOIN class_info ON class.id = class...	3 row(s) returned	0.016 sec / 0.000 sec
✓ 10	14:34:06	SELECT class.name, class_info.address FROM class INNER ...	3 row(s) returned	0.000 sec / 0.000 sec
✓ 11	14:35:24	SELECT * FROM class NATURAL JOIN class_info LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec

Program:

```
47      -- Step 8: LEFT OUTER JOIN
48 •    SELECT *
49      FROM class
50      LEFT OUTER JOIN class_info
51      ON class.id = class_info.id;
```

Output:

The screenshot displays a database IDE interface. At the top, a 'Result Grid' shows the output of a query. It has columns 'id', 'name', 'id', and 'address'. The data rows are as follows:

	id	name	id	address
▶	1	Tejaswini	1	DELHI
	2	Praneeth	2	MUMBAI
	3	Sahasra	3	CHENNAI
	4	Pavani	NULL	NULL
	5	Nikila	NULL	NULL

Below the result grid, the 'Action Output' tab is selected, showing a log of database actions. The log includes the following entries:

#	Time	Action	Message	Duration / Fetch
✓ 7	14:31:20	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
✓ 8	14:32:14	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
✓ 9	14:33:09	SELECT * FROM class INNER JOIN class_info ON class.id = class...	3 row(s) returned	0.016 sec / 0.000 sec
✓ 10	14:34:06	SELECT class.name, class_info.address FROM class INNER ...	3 row(s) returned	0.000 sec / 0.000 sec
✓ 11	14:35:24	SELECT * FROM class NATURAL JOIN class_info LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
✓ 12	14:36:59	SELECT * FROM class LEFT OUTER JOIN class_info ON class.id ...	5 row(s) returned	0.016 sec / 0.000 sec

Program:

```
52  -- Step 9: LEFT JOIN with NULL
53  •  SELECT *
54      FROM class
55      LEFT JOIN class_info
56      ON class.id = class_info.id
57      WHERE class_info.id IS NULL;
```

Output:

The screenshot displays a database IDE interface. At the top, a 'Result Grid' shows the results of the query. It has columns 'id', 'name', 'id', and 'address'. The first row shows '4', 'Pavani', 'NULL', and 'NULL'. The second row shows '5', 'Nikita', 'NULL', and 'NULL'. Below the result grid, there is a vertical toolbar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. At the bottom, an 'Output' panel shows a log of database actions. The log includes the query being executed and the number of rows returned. The query is: 'SELECT * FROM class LEFT JOIN class_info ON class.id = class_info.id WHERE class_info.id IS NULL;'. The log shows that 2 rows were returned.

#	Time	Action	Message	Duration / Fetch
8	14:32:14	SELECT * FROM class CROSS JOIN class_info LIMIT 0, 1000	25 row(s) returned	0.000 sec / 0.000 sec
9	14:33:09	SELECT * FROM class INNER JOIN class_info ON class.id = class...	3 row(s) returned	0.016 sec / 0.000 sec
10	14:34:06	SELECT class.name, class_info.address FROM class INNER ...	3 row(s) returned	0.000 sec / 0.000 sec
11	14:35:24	SELECT * FROM class NATURAL JOIN class_info LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
12	14:36:59	SELECT * FROM class LEFT OUTER JOIN class_info ON class.id ...	5 row(s) returned	0.016 sec / 0.000 sec
13	14:37:47	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	2 row(s) returned	0.000 sec / 0.000 sec

Program:

```
58      -- Step 10: RIGHT OUTER JOIN
59  •    SELECT *
60      FROM class
61      RIGHT OUTER JOIN class_info
62      ON class.id = class_info.id;
```

Output:

The screenshot shows a database query tool interface. At the top, there's a 'Result Grid' tab with a table containing 5 rows and 4 columns: 'id', 'name', 'id', and 'address'. The data rows are: (1, Tejaswini, 1, DELHI), (2, Praneeth, 2, MUMBAI), (3, Sahasra, 3, CHENNAI), (NULL, NULL, 7, NOIDA), and (NULL, NULL, 8, PANIPAT). To the right of the table are icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. Below the table, there's a 'Result 8' tab and buttons for 'Read Only', 'Context Help', and 'Snippets'. The 'Output' section at the bottom shows a list of actions with their execution times, messages, and durations. The actions include an inner join, a select with aliases, a natural join with a limit, a left outer join, a left join, and the current right outer join.

#	Time	Action	Message	Duration / Fetch
9	14:33:09	SELECT * FROM class INNER JOIN class_info ON class.id = class...	3 row(s) returned	0.016 sec / 0.000 sec
10	14:34:06	SELECT class.name, class_info.address FROM class INNER ...	3 row(s) returned	0.000 sec / 0.000 sec
11	14:35:24	SELECT * FROM class NATURAL JOIN class_info LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
12	14:36:59	SELECT * FROM class LEFT OUTER JOIN class_info ON class.id ...	5 row(s) returned	0.016 sec / 0.000 sec
13	14:37:47	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	2 row(s) returned	0.000 sec / 0.000 sec
14	14:39:35	SELECT * FROM class RIGHT OUTER JOIN class_info ON class.i...	5 row(s) returned	0.000 sec / 0.000 sec

Program:

```
63      -- Step 11: RIGHT JOIN with NULL
64      SELECT *
65      FROM class
66      RIGHT JOIN class_info
67      ON class.id = class_info.id
68      WHERE class.id IS NULL;
```

Output:

Result Grid				
Filter Rows:				
	id	name	id	address
▶	NULL	NULL	7	NOIDA
	NULL	NULL	8	PANIPAT

15 14:40:13 SELECT * FROM class RIGHT JOIN class_info ON class.id = class... 2 row(s) returned 0.000 sec / 0.000 sec

Program:

```
69      -- Step 12: FULL OUTER JOIN
70      SELECT *
71      FROM class
72      LEFT JOIN class_info
73      ON class.id = class_info.id
```

Output:

Result Grid				
Filter Rows:				
	id	name	id	address
▶	1	Tejaswini	1	DELHI
	2	Praneeth	2	MUMBAI
	3	Sahasra	3	CHENNAI
	4	Pavani	NULL	NULL
	5	Nikila	NULL	NULL
	NULL	NULL	7	NOIDA
	NULL	NULL	8	PANIPAT

✓ 16 14:41:44 SELECT * FROM class LEFT JOIN class_info ON class.id = class_i... 7 row(s) returned 0.000 sec / 0.000 sec

Program:

```
75 UNION
76 SELECT *
77 FROM class
78 RIGHT JOIN class_info
79 ON class.id = class_info.id;
```

Output:

The screenshot shows a database management tool interface. At the top, there's a 'Result Grid' tab with a table containing 5 columns: 'id', 'name', 'id', and 'address'. The table has 8 rows of data. Below the table, there's a 'Result 11' tab with a 'Read Only' button and 'Context Help' and 'Snippets' links. The 'Output' section shows a list of actions performed, including a 'SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...' query that returned 7 rows.

#	Time	Action	Message	Duration / Fetch
✓ 12	14:36:59	SELECT * FROM class LEFT OUTER JOIN class_info ON class.id ...	5 row(s) returned	0.016 sec / 0.000 sec
✓ 13	14:37:47	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 14	14:39:35	SELECT * FROM class RIGHT OUTER JOIN class_info ON class.i...	5 row(s) returned	0.000 sec / 0.000 sec
✓ 15	14:40:13	SELECT * FROM class RIGHT JOIN class_info ON class.id = class...	2 row(s) returned	0.000 sec / 0.000 sec
✓ 16	14:41:44	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	7 row(s) returned	0.000 sec / 0.000 sec
✓ 17	14:42:17	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	7 row(s) returned	0.016 sec / 0.000 sec

Program:

```
80 -- Step 13: FULL OUTER JOIN Unmatched Records
81 • SELECT *
82 FROM class
83 LEFT JOIN class_info
84 ON class.id = class_info.id
85 WHERE class_info.id IS NULL
```

Output:

Result Grid			
Filter Rows:			
Export: Wrap Cell Content:			
id	name	id	address
4	Pavani	NULL	NULL
5	Nikila	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

#	Time	Action	Message	Duration / Fetch
13	14:37:47	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	2 row(s) returned	0.000 sec / 0.000 sec
14	14:39:35	SELECT * FROM class RIGHT OUTER JOIN class_info ON class.i...	5 row(s) returned	0.000 sec / 0.000 sec
15	14:40:13	SELECT * FROM class RIGHT JOIN class_info ON class.id = class...	2 row(s) returned	0.000 sec / 0.000 sec
16	14:41:44	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	7 row(s) returned	0.000 sec / 0.000 sec
17	14:42:17	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	7 row(s) returned	0.016 sec / 0.000 sec
18	14:43:14	SELECT * FROM class LEFT JOIN class_info ON class.id = class_i...	4 row(s) returned	0.000 sec / 0.000 sec

Program:

```
87     UNION
88     SELECT *
89     FROM class
90     RIGHT JOIN class_info
91     ON class.id = class_info.id
92     WHERE class.id IS NULL;
```

Output:

Result Grid				
Filter Rows:				
	id	name	id	address
▶	4	Pavani	NULL	NULL
	5	Nikila	NULL	NULL
	NULL	NULL	7	NOIDA
	NULL	NULL	8	PANIPAT

21 14:45:46 SELECT * FROM class LEFT JOIN class_info ON class.id = class_i... 4 row(s) returned 0.000 sec / 0.000 sec

Program:

```
93      -- Step 14: Create Set Operation Tables
94  ●  CREATE TABLE first_table(
95      id INT,
96      name VARCHAR(30)
97  );
98
99  ●  CREATE TABLE second_table(
100      id INT,
101      name VARCHAR(30)
102  );
103      -- Step 15: Insert Set Operation Data
104  ●  INSERT INTO first_table VALUES
105      (1,'Tejaswini'),
106      (2,'Sree');
107
108  ●  INSERT INTO second_table VALUES
109      (2,'Sree'),
110      (3,'Virat');
```

Output:

✓	32 14:51:24	CREATE TABLE first_table(id INT, name VARCHAR(30))	0 row(s) affected	0.125 sec
✓	33 14:51:28	CREATE TABLE second_table(id INT, name VARCHAR(30))	0 row(s) affected	0.032 sec
✓	34 14:51:30	INSERT INTO first_table VALUES (1,'Tejaswini'), (2,'Sree')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.015 sec
✓	35 14:51:32	INSERT INTO second_table VALUES (2,'Sree'), (3,'Virat')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec

Program:

```

111      -- Step 16: UNION
112      SELECT *
113      FROM first_table
114      UNION
115      SELECT *
116      FROM second_table;

```

Output:

Result Grid			Filter Rows:
	id	name	
▶	1	Tejaswini	
	2	Sree	
	3	Virat	

37 14:52:51 SELECT * FROM first_table UNION SELECT * FROM second_table 3 row(s) returned 0.000 sec / 0.000 sec

Program:

```

117      -- Step 17: UNION Names
118      SELECT name
119      FROM first_table
120      UNION
121      SELECT name
122      FROM second_table;

```

Output:

Result Grid		Filter Rows:
	name	
▶	Tejaswini	
	Sree	
	Virat	

38 14:53:57 SELECT name FROM first_table UNION SELECT name FROM se... 3 row(s) returned 0.000 sec / 0.000 sec

Program:

```

123      -- Step 18: UNION ALL
124      SELECT *
125      FROM first_table
126      UNION ALL
127      SELECT *
128      FROM second_table;

```

Output:

Result Grid			Filter Row
	id	name	
▶	1	Tejaswini	
	2	Sree	
	2	Sree	
	3	Virat	

39 14:54:46 SELECT * FROM first_table UNION ALL SELECT * FROM second... 4 row(s) returned 0.000 sec / 0.000 sec

Program:

```

129      -- Step 19: Count UNION ALL Records
130      SELECT COUNT(*)
131      FROM (
132          SELECT * FROM first_table
133
134          UNION ALL
135
136          SELECT * FROM second_table
137      ) AS A;

```

Output:

Result Grid	
	COUNT(*)
▶	4

41 14:56:12 SELECT COUNT(*) FROM (SELECT * FROM first_table UNI... 1 row(s) returned 0.000 sec / 0.000 sec

Program:

```

138      -- Step 20: INTERSECT
139      SELECT *
140      FROM first_table
141
142      INTERSECT
143      SELECT *
144      FROM second_table;

```

Output:

The screenshot shows a database IDE interface. At the top, a SQL query is displayed, which is an SQL query using the `INTERSECT` operator to find common rows between `first_table` and `second_table`. Below the query, the 'Output' section is visible, showing a table with columns: `#`, `Time`, `Action`, `Message`, and `Duration / Fetch`. The table contains six rows of execution logs, all marked with a green checkmark, indicating successful execution. The last row shows the execution of the `INTERSECT` query, returning 1 row(s).

#	Time	Action	Message	Duration / Fetch
✓ 37	14:52:51	SELECT * FROM first_table UNION SELECT * FROM second_table	3 row(s) returned	0.000 sec / 0.000 sec
✓ 38	14:53:57	SELECT name FROM first_table UNION SELECT name FROM se...	3 row(s) returned	0.000 sec / 0.000 sec
✓ 39	14:54:46	SELECT * FROM first_table UNION ALL SELECT * FROM second...	4 row(s) returned	0.000 sec / 0.000 sec
✓ 40	14:55:42	SELECT COUNT(*) FROM (SELECT * FROM first_table UNI...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 41	14:56:12	SELECT COUNT(*) FROM (SELECT * FROM first_table UNI...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 42	14:57:05	SELECT * FROM first_table INTERSECT SELECT * FROM secon...	1 row(s) returned	0.000 sec / 0.000 sec

Program:

```

145      -- Step 21: INTERSECT Names
146  ●    SELECT name
147      FROM first_table
148
149      INTERSECT
150
151  ✖    SELECT name
152      FROM second_table;
153      -- Step 22: MINUS
154  ●    SELECT f.*
155      FROM first_table f
156  ○    WHERE NOT EXISTS (
157          SELECT 1
158          FROM second_table s
159          WHERE s.id = f.id
160          AND s.name = f.name
161      );

```

Output:

The screenshot shows a database IDE interface. At the top, there's a 'Result Grid' tab with a table containing two rows: 'name' and 'Sree'. Below this, there's a vertical toolbar with icons for 'Form Editor', 'Field Types', and 'Query Stats'. At the bottom, there's an 'Output' section with a dropdown menu set to 'Action Output'. This section displays a log of database actions, including SELECT statements and their execution times and messages.

#	Time	Action	Message	Duration / Fetch
38	14:53:57	SELECT name FROM first_table UNION SELECT name FROM se...	3 row(s) returned	0.000 sec / 0.000 sec
39	14:54:46	SELECT * FROM first_table UNION ALL SELECT * FROM second...	4 row(s) returned	0.000 sec / 0.000 sec
40	14:55:42	SELECT COUNT(*) FROM (SELECT * FROM first_table UNI...	1 row(s) returned	0.000 sec / 0.000 sec
41	14:56:12	SELECT COUNT(*) FROM (SELECT * FROM first_table UNI...	1 row(s) returned	0.000 sec / 0.000 sec
42	14:57:05	SELECT * FROM first_table INTERSECT SELECT * FROM secon...	1 row(s) returned	0.000 sec / 0.000 sec
43	14:57:50	SELECT name FROM first_table INTERSECT SELECT name FR...	1 row(s) returned	0.000 sec / 0.000 sec

Program:

```

153      -- Step 22: MINUS
154  ●   SELECT f.*
155      FROM first_table f
156  ○   WHERE NOT EXISTS (
157      |       SELECT 1
158      |       FROM second_table s
159      |       WHERE s.id = f.id
160      |       AND s.name = f.name
161      |   );
162      -- Step 23: MINUS Names
163  ●   SELECT f.name
164      FROM first_table f
165  ○   WHERE NOT EXISTS (
166      |       SELECT 1
167      |       FROM second_table s
168      |       WHERE s.name = f.name
169      |   );

```

Output:

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	id	name
▶	1	Tejaswini

Result Grid

Form Editor

Field Types

Query Stats

first_table 25 ×

Read Only

Context Help

Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 40	14:55:42	SELECT COUNT(*) FROM (SELECT * FROM first_table UNI...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 41	14:56:12	SELECT COUNT(*) FROM (SELECT * FROM first_table UNI...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 42	14:57:05	SELECT * FROM first_table INTERSECT SELECT * FROM secon...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 43	14:57:50	SELECT name FROM first_table INTERSECT SELECT name FR...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 44	14:58:12	SELECT name FROM first_table INTERSECT SELECT name FR...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 45	14:58:34	SELECT f.* FROM first_table f WHERE NOT EXISTS (SELECT...	1 row(s) returned	0.000 sec / 0.000 sec